RAG Q&A assistant over Python documentation: report

Corpus: CPython 3.12.7 library docs (json, string, datetime, urllib.request) and requests 2.32.3 (api, quickstart, advanced). Models: qwen2.5:3b-instruct (Q4_K_M) and nomic-embed-text, served by Ollama. Every number below comes from `python evaluate.py`, offline, on one laptop (i5-9300H, 4 cores, 8 GB, GTX 1650). Quality numbers: `config.yaml`, where Ollama used the GPU. Target latency: `config.cpu.yaml`, CPU only, 4 threads.

1. Architecture and the reasoning behind it

```
ingest.py   reST files -> structure-aware chunks -> nomic-embed-text -> SQLite (text, metadata, float32 vectors)

/query -> retrieve -> grade --sufficient--> generate --answer--> verify --pass--> finish            [answered]
            ^         |                       |                   |--fail, bounded--> generate (told why)
            |         |--insufficient,        |--NOT_IN_DOCS-->    |--fail, limit----> finish_unverified
            |         |  retries left         refuse               [refused_unverified, failure disclosed]
            +-- recover (rewrite query; next retrieve skips every passage already seen)
                      |--insufficient, no retries left--> refuse [retry_limit / refused_no_context]
```

Retrieval is hybrid: BM25 over the chunk text plus a section/entry prefix, and cosine similarity over nomic embeddings, fused with weighted reciprocal rank fusion (BM25 weight 2, dense 1, k=60). Reference docs are full of exact tokens (`%j`, `sort_keys`, `r.json()`) that a 137M embedding model cannot separate; BM25 can. Dense search still finds paraphrased questions. Vectors live in SQLite with numpy search: 582 chunks need no ANN index, and one file is easy to rebuild and ship offline.

Grading is per passage and evidence-based (section 4). **Generation** sees only retained passages, labelled P1..Pn, plus the grader's evidence. **Verification** is two-stage: deterministic checks first (citations must be retained passages; every backticked identifier must appear in a cited passage), then an LLM check that each claim is supported. Every model call is constrained by a JSON schema through Ollama structured outputs, so replies always parse.

Rejected: (1) one grader call over all 8 passages. It was my first design; with a 3B model it kept the gold passage on only 31% of answerable questions (it often listed the right passage as relevant and still said "insufficient"). (2) A yes/no relevance grader per passage: better (gold kept on 10 of 15 retrieved), but it still said "no" to passages that plainly answer. (3) Widening retrieval to 16 passages on the last retry: the prompt overflowed the 4,096-token context and Ollama silently cut the instructions. (4) A cross-encoder reranker: one more model to ship, and it does not solve the grader problem. (5) Chroma/FAISS: unnecessary at this size.

2. State schema

Field	Written by	Read by	Why it exists
`question`	caller	all nodes	The original question; the grader and verifier always judge against it, never against a rewrite.
`query`	caller, recover	retrieve, recover, refuse	The current search string. Separate from `question` so a rewrite cannot change what is being answered.
`retries`	recover	route_after_grade, refuse	Bounds the corrective loop (`max_retries`=2); returned to the caller.
`candidates`	retrieve	grade	This attempt's top-8, excluding everything seen before.
`retrieved`	retrieve	retrieve, refuse	All chunk ids seen, in rank order. Lets a retry search deeper instead of re-grading; returned as `retrieved`.
`sufficient`	grade	route_after_grade, refuse	The explicit sufficiency decision: at least one passage carries verified evidence.
`retained`	grade	generate, verify	The only passages the generator may see and cite.
`evidence`	grade	generate	The text the grader copied from each retained passage: why it was kept, and a pointer for the generator.
`answer`	generate, refuse, finish_unverified	route_after_generate, verify, finish_unverified	Empty after generate means "docs do not answer", which routes to refuse.
`citations`	generate, refuse	verify	Chunk ids the answer relies on; must be a subset of `retained`.
`verified`	verify	route_after_verify, refuse	Verification result; `null` if verification was never reached.
`verify_failures`	verify	route_after_verify, generate	Bounds regeneration (`max_verify_retries`=1) and tells generate to include the failure reason.

`note`	grade, recover, generate, verify	generate	Last decision's reason: the trace line, and the failure reason fed back on regeneration.
`refused`, `termination`	refuse, finish, finish_unverified	caller	The outcome. Only the three terminal nodes write them, so every END has both set.

How each path ends. `answered`: verification passed. `refused_unverified`: verification failed twice; the answer is returned prefixed with "Verification failed for this answer" and `verified=false`. `refused_no_answer`: the grader kept passages but the generator found no answer in them. `retry_limit`: no evidence after 3 attempts (24 distinct passages). `refused_no_context`: nothing retrieved at all. `graph_limit`: LangGraph's recursion limit (25 steps, longest legal path 15) as an independent safety net; never reached. The API adds `timeout` (HTTP 504) and model-down (503). Every loop has a counter in state with a bound from config, so the worst case is fixed: 3×8 grader calls + 2 rewrites + 2 generations + 2 verifications = 30 model calls. Tests drive a scripted model through every path, including a model that always rewrites to an empty query.

3. Chunking and retrieval

The parser reads reStructuredText directly: it is the source the HTML is built from, and it keeps the structure (section underlines, `.. function::` directives, grid tables, `.. versionchanged::`) that HTML flattens. Three chunk types: **entry** (one API directive with its whole body; an entry over 2,000 characters is split by paragraph and every part repeats the signature), **table** (a grid table with its caption; a table over 1,200 characters is split only between rows and every piece repeats the caption and the header row), and **prose** (paragraphs of one section, packed to 1,200 characters, never split mid-paragraph). Every chunk records file, section path, line range, entry name and its version notes. 582 chunks: 291 entry, 250 prose, 41 table; 93 carry a version note. Citations are `file:start-end (section path)`.

Tables are read as rows. A grid row spans several lines between | borders. For embedding, BM25 and every prompt, each row is rewritten as `Header: cell; ...`. The stored text, line numbers and citations stay on the source.

Table example: datetime.rst:2425-2427

```
The following is a list of all the format codes that the 1989 C
standard requires, and these work on all platforms ...
+-----------+--------------------------------+--------------------+
| Directive | Meaning                        | Example            |
+===========+================================+====================+
| ``%j``    | Day of the year as a           | 001, 002, ..., 366 |
|           | zero-padded decimal number.    |                    |
as the model sees it:
Directive: ``%j``; Meaning: Day of the year as a zero-padded
decimal number.; Example: 001, 002, ..., 366; Notes: \(9)
```

Function-entry example: json.rst:137-206, part 2

```
function:: dump(obj, fp, *, skipkeys=False,
ensure_ascii=True,
  ..., indent=None, separators=None,
default=None,
  sort_keys=False, **kw)  (part 2)
   .. versionchanged:: 3.2
      Allow strings for *indent* in addition to
integers.
   If specified, *separators* should be an
   ``(item_separator, key_separator)`` tuple.
...
metadata: entry=dump, chunk_type=entry,
version_notes=
 [versionchanged 3.2 ..., versionchanged 3.4
..., versionchanged 3.6 ...]
```

What I measured. Rank of the best gold chunk for the 16 answerable questions (gold matching: a chunk matches when it is from the gold file and contains the gold quote after whitespace normalisation; a test confirms every quote sits inside one chunk).

Configuration	hit@5	hit@8	Gold ranks q01..q16
Before: format-code table as one 8,068-char chunk, plain RRF	11/16	14/16	2 1 1 1 1 1 1 1 8 6 9 8 16 2 1 3
Tables split by rows, BM25 weight 2	13/16	15/16	3 1 1 1 1 1 1 1 5 4 8 8 14 1 1 3
+ rows linearised, weight 1	13/16	15/16	3 1 1 1 1 1 1 1 8 1 5 7 11 2 1 3
+ rows linearised, weight 2 (shipped)	**14/16**	**16/16**	3 1 1 1 1 1 1 1 5 1 3 8 8 1 1 3
+ rows linearised, weight 3	15/16	16/16	3 1 1 1 1 1 1 1 4 1 3 8 5 1 1 3

The weight was chosen on these same 16 questions; there is no held-out set, so treat the difference between 2 and 3 as noise. I kept 2, the smaller change from plain fusion. Dense alone ranks the %p row 95th: the embedding cannot tell format codes apart.

4. Grading, recovery, refusal and verification

Sufficiency. The grader sees one passage at a time and must copy, word for word, the sentence or table row that answers the question, or return an empty string. Code then checks that the first 60 characters of the copy appear in the passage (after folding whitespace and reST markup). A passage is retained only if it passes, and context is sufficient when at least one passage is retained. Small models are better at extracting than at judging, and the check means an answer from

memory or a paraphrase cannot make context sufficient (a test feeds a fluent but uncopied claim about HTTP/2 and expects a refusal). Grading stops once 3 passages carry evidence.

Recovery. When nothing carries evidence: rewrite the query (if the model repeats it, fall back to the question's content words), then retrieve the next 8 passages not yet seen. Two retries, so up to 24 distinct passages. **Refusal** names the query and the number of passages checked. **Verification** as in section 1. On failure the generator is told why and tries once more; if it fails again, the answer goes out with the failure disclosed.

Node	Visits / request	Mean s, GTX 1650	Mean s, CPU 4 threads	Model calls per visit
retrieve	1.20	0.4	0.6	0 (one query embedding)
grade	1.20	13.7	53.4	3–8, one per passage until 3 carry evidence
recover	0.20	0.9	3.1	1
generate	0.95	4.5	26.2	1
verify	0.70	1.2	4.6	0–1 (LLM only if the deterministic checks pass)

From `data/traces.jsonl` and `data/latency/cpu_4thread/traces.jsonl`. Grading is about three quarters of request time on both: the price of one passage per call. Verification is cheap because it sees only the cited passages.

5. Evaluation results

Quality numbers come from the run in `data/results.jsonl` (`config.yaml`; on this machine Ollama placed both models on a GTX 1650, offline). The CPU-only run (`config.cpu.yaml`, 4 threads, `data/latency/cpu_4thread/`) is the target-hardware latency measurement; its quality numbers are close but not identical (CPU and GPU arithmetic differ, so a 3B model can choose differently): unanswerable refused 4/4, answerable wrongly refused 4/16; outcomes differ from the GPU run on q10, q16.

Category	n	Gold in retrieved (hit@8)	Gold in retained
single_passage	9	9/9	8/9
table_lookup	4	4/4	4/4
multi_document	2	2/2	2/2
version_dependent	1	1/1	0/1
All answerable	16	**16/16**	**14/16**

hit@8 counts the first attempt's 8 passages only. "Retained" means a gold chunk survived grading on any attempt. For multi_document, one gold passage counts as a hit; q15 retained only the json half.

Unanswerable refused	Answerable wrongly refused	Terminations (20)	Latency, GTX 1650: median / p95	Latency, CPU 4 threads (target): median / p95
4/4	**2/16** (q06, q08)	14 answered, 5 refused_no_answer, 1 retry_limit	20.9 s / 35.9 s	**85.1 s / 176.3 s**

Groundedness (manual pass by the author, claim by claim against the cited passages). All 14 answered questions passed the automatic checks and the LLM verifier. The manual pass is stricter:

Verdict	Questions	Detail
Correct, every claim supported by a cited passage	q02 q03 q05 q07 q09 q10 q11 q12 (8)	Table answers cite the exact row, e.g. q10 → `datetime.rst:2425-2427`.
Correct, but a claim is not in the cited passage	q01 q13 (2)	q01: "useful for regression tests" is true (json.rst:447) but not in the cited intro chunk. q13: the AM/PM row was retained but the answer cites a neighbouring note.
Supported but incomplete	q04 q14 (2)	q04 omits mounting the `HTTPAdapter`; q14 omits that requests also has a separate read timeout.
Partly unsupported	q15 (1)	The requests half ("requests raises a JSONDecodeError") cites only json.rst passages.
Wrong	q16 (1)	States the opposite of the deprecation note, and passed verification. See below.

So 8 of 14 answers are fully grounded, 4 more are right with a citation or completeness defect, and 2 contain a claim their citations do not support. The verifier's pass rate (14/14) overstates groundedness; that is the single most important limitation of this system.

Three traced failures

q16, version_dependent: wrong answer, verified. Retrieved: `urllib.request#0003`, `#0002`, `#0004` (gold: the `.. deprecated:: 3.6` note) at ranks 1–3. Grader: copied evidence only from `#0002` ("The optional *cafile* and *capath* parameters specify a set of trusted CA certificates..."); for `#0004` it returned nothing. Generator: from a passage that describes the parameters and says nothing about deprecation, it concluded they are "still recommended". Verifier: the LLM check accepted it. *Single fix:* `urlopen` is one entry split into three parts; show the generator every part's `version_notes` (already in chunk metadata) whenever any part is retained.

q08, single_passage: wrongly refused. Retrieved: the gold chunk `datetime#0046` at rank 1. Grader: no evidence from any of 8 passages; after a rewrite, 8 more, one irrelevant sentence retained. Generator: NOT_IN_DOCS, so refused. The answer is the one-line example `date.fromisoformat('20191204')`, and the grader prompt says "copy prose, not a code example". I added that rule to stop the model copying a whole code block (which tripped Ollama's repeat limit on q04); it also blocks this. *Single fix:* allow one line of code as evidence and forbid only multi-line blocks.

q15, multi_document: half unsupported. Retrieved: requests `quickstart#0008` (gold) at rank 1 and json gold chunks at ranks 3 and 8. Grader: no evidence from `quickstart#0008`, then 3 json passages carried evidence, so grading stopped after 4 of 8. Generator: answered both halves from json passages alone. The verifier did not notice the requests claim had no requests citation. *Single fix:* when the question names two libraries, do not stop grading until each has a retained passage.

Also wrong: q06 (refused). The grader retained the gold passage with the right evidence ("the success of the call to `r.json()` does **not** indicate the success of the response") and the generator still said NOT_IN_DOCS, a 3B model being over-cautious on a yes/no question.

On q19 ("What is `Response.elapsed`?"): the local `api.rst` documents `Response` with `.. autoclass::`, which Sphinx fills from docstrings when it builds the website. The rendered page does describe `elapsed`; the reST copy this assistant holds does not, so refusal is correct for this corpus. Autodoc-generated content is a known gap in using the reST sources.

6. What the local setup cost, and what I would change

Quality. A 3B model is the binding constraint, not retrieval: the gold passage is in the top 8 for 16/16 answerable questions, yet three answers are incomplete and one is wrong. A 3B grader cannot judge sufficiency over 8 passages at once, which forced the one-passage-per-call design. It is weak at negation and synthesis (q16), and it does not notice when half of a two-part question is uncovered (q15). The LLM verifier passes what the generator writes too readily; the deterministic checks caught more than the model did.

Time. Grading is 71% (CPU) and 74% (GPU) of request time: up to 8 model calls, each reading a passage from scratch. On the target (4 CPU threads), prompt reading runs at about 54 tokens/s and generation at about 7 tokens/s, so one grader call costs 10–16 s. What I cut to get closer: grading stops after 3 passages with evidence, retries never re-grade a passage, prompts are capped at 4,096 tokens, replies are capped at 160 tokens and constrained to a schema, and the model is called on 127.0.0.1 rather than `localhost`. That last one is worth about 2 s per call on Windows, ~20 s per request. Measured on 4 CPU threads: median 85 s, p95 176 s per request, so the 20 s budget is missed by a factor of about 4. The same code on the GTX 1650 has a median of 20.9 s.

With a GPU (a datacenter card, not this laptop's GTX 1650): grade the 8 passages as one batched request (Ollama `num_parallel`/vLLM), which removes most of the grading time, and move to a 7B–8B model for the generator and verifier, where most remaining errors are. **With two more weeks:** fix the three traced failures (version notes travel with every part of an entry; one-line code examples count as evidence; per-document coverage before grading stops); a verifier that checks each clause of the answer against one retained passage; a held-out question set so the fusion weight and prompts are not tuned on the evaluation questions; and a second annotator for the groundedness pass.